Bootstrap-Consistency Regularization: Training Neural Networks for Prediction Stability*

Gaurav Sood†

April 5, 2026

Abstract

Neural networks exhibit substantial prediction variability when retrained on bootstrap samples of the same dataset, undermining reliability in deployment scenarios requiring consistent decision boundaries. We propose *bootstrap-consistency regularization* (BCR), a training procedure that directly minimizes prediction variance across data resamples. Our method trains multiple shadow copies of a network simultaneously, each receiving Poisson-weighted losses that simulate bootstrap resampling, while penalizing disagreement between their predictions on identical inputs. We provide a first-order sensitivity analysis showing that bootstrap prediction variance depends on the gradient-Hessian-gradient product, which our variance penalty directly targets. Empirical evaluation on four tabular datasets demonstrates 22–65% reductions in bootstrap prediction variance compared to standard training, with accuracy degradation typically under one percentage point. Comprehensive experiments across 180 hyperparameter configurations reveal that BCR provides the largest benefits in low-sample, high-dimensional settings (45–65% improvement) and remains effective under label noise (24–38% improvement) and correlated features (14–31% improvement). BCR provides strong stability improvements on classification tasks, though simpler methods (bagging, weight decay) may be equally effective on some regression tasks. Unlike existing approaches that target indirect proxies such as weight-space curvature, BCR directly optimizes the quantity of practical interest: prediction consistency under data resampling.

1 Introduction

The deployment of neural networks in production systems requires not only predictive accuracy but also consistency across model updates. When a model is retrained on fresh samples from the same distribution, predictions on identical inputs should remain stable within the

*https://github.com/finite-sample/consistentshade.

†Gaurav can be reached at gsood07@gmail.com

bounds justified by sampling uncertainty. However, neural networks commonly exhibit substantial *refit variance*—the phenomenon whereby predictions vary significantly when models are trained on different bootstrap samples of the training data (Efron, 1979).

This instability poses significant challenges across multiple domains. In production machine learning systems, model updates may reverse binary classifications on borderline cases, creating inconsistent user experiences. Scientific applications require stable models for fair method comparisons and reliable bootstrap-based confidence intervals. Regulated industries face compliance issues when prediction variability across training runs triggers audit procedures.

Existing approaches to neural network stability primarily target indirect proxies for the desired behavior. Sharpness-Aware Minimization (Foret et al., 2021) and related methods penalize weight-space curvature under the assumption that flatter minima correspond to more stable predictions. Stochastic regularization techniques such as R-Drop (Liang et al., 2021) control prediction consistency under network noise but do not address data resampling variance. Teacher-student methods like Mean Teacher (Tarvainen and Valpola, 2017) stabilize optimization dynamics while remaining agnostic to bootstrap variance. Standard bagging (Breiman, 1996) reduces variance through ensembling but does not encourage individual models to be stable.

We propose *bootstrap-consistency regularization* (BCR), a fundamentally different approach that directly minimizes prediction variance across data resamples. Our key contributions are:

1. **Estimand**: We clearly define bootstrap prediction instability as the target quantity of interest—the variance of predictions across models trained on bootstrap resamples of the data.

2. **Training objective**: We augment the standard empirical risk with an explicit instability penalty, creating a principled trade-off between accuracy and stability.

3. **Algorithm**: We introduce a tractable stochastic surrogate that approximates full-retraining instability through mini-batch Poisson weighting, enabling efficient joint optimization.

4. **Theory**: We provide first-order sensitivity analysis showing that bootstrap variance depends on gradient sensitivity through the loss Hessian, justifying why direct variance penalties target the right quantity.

5. **Empirics**: We demonstrate that BCR reduces actual full-retraining instability by 22–50% across four datasets, with competitive performance relative to natural baselines on the stability-accuracy Pareto frontier.

2 Related Work

Our approach differs fundamentally from existing stability methods in directly targeting prediction variance under data resampling rather than indirect proxies.

Sharpness-based methods such as Sharpness-Aware Minimization (Foret et al., 2021) and Entropy-SGD (Chaudhari et al., 2017) seek flatter loss surfaces under the hypothesis that such minima correspond to more stable predictions. Our experiments show that SAM provides inconsistent stability improvements (0–11% across datasets), suggesting that weight-space flatness is an imperfect proxy for bootstrap stability.

Stochastic consistency methods like R-Drop (Liang et al., 2021) enforce agreement between predictions under different dropout masks, addressing network stochasticity but not data resampling variance. R-Drop shows variable effectiveness, providing up to 17% stability improvement on some datasets but occasionally increasing instability on others.

Bagging and ensembles (Breiman, 1996) reduce variance through model averaging. Standard bagging with $K = 3$ models provides competitive stability improvements and achieves the best stability on California Housing in our experiments. BCR outperforms bagging on classification tasks (Adult, German Credit) but not on California Housing regression, suggesting a pattern: BCR may provide greater benefits when prediction variance stems from model specification uncertainty (which regularization can address) rather than irreducible noise in the target variable (which it cannot). California Housing's continuous target with substantial measurement noise may favor bagging's averaging approach over BCR's attempt to stabilize individual models.

Teacher-student approaches including Mean Teacher (Tarvainen and Valpola, 2017) stabilize training dynamics through exponential moving averages of model weights. While effective for reducing optimization noise, these methods do not explicitly address variability under data resampling.

Distributionally robust optimization methods such as χ^2-DRO (Duchi and Namkoong, 2021) penalize loss variance across data subsets. However, loss variance does not directly correspond to prediction variance—models may achieve similar loss values while producing substantially different predictions.

Influence functions (Koh and Liang, 2017) provide a theoretical framework for understanding prediction sensitivity to training data. Our first-order analysis (Equation 5) builds on this framework to explain why direct variance penalties are well-motivated.

3 Method

3.1 Problem Formulation

Consider a training dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ drawn from distribution P, and a parameterized model $f_\theta : \mathcal{X} \to \mathcal{Y}$. For any test input $x \in \mathcal{X}$, we define the *bootstrap prediction variance* as:

$$\sigma^2_{\text{boot}}(x) = \mathbb{E}_{\mathcal{D}' \sim \text{Boot}(\mathcal{D})} \left[(\hat{f}_{\mathcal{D}'}(x) - \mathbb{E}[\hat{f}_{\mathcal{D}'}(x)])^2 \right] \tag{1}$$

where $\hat{f}_{\mathcal{D}'}$ denotes the model obtained by training on bootstrap sample $\mathcal{D}'$, and $\text{Boot}(\mathcal{D})$ represents the bootstrap distribution over datasets of size n sampled with replacement from $\mathcal{D}$.

Our objective combines standard empirical risk minimization with explicit bootstrap variance regularization:

$$\min_{\theta} \mathbb{E}_{(x,y)\sim P}[\ell(f_\theta(x), y)] + \lambda \cdot \mathbb{E}_{x\sim P_X}[\sigma^2_{\text{boot}}(x)] \tag{2}$$

where ℓ is a loss function, P_X is the marginal distribution of inputs, and $\lambda > 0$ controls the regularization strength.

3.2 First-Order Sensitivity Analysis

To understand what drives bootstrap prediction variance, we analyze the sensitivity of predictions to training data perturbations. Following the influence function framework (Koh and Liang, 2017), consider an infinitesimal upweighting of training point (x_i, y_i) by ϵ. The change in optimal parameters is:

$$\frac{d\theta^*}{d\epsilon_i} = -H^{-1}\nabla_\theta \ell(f_\theta(x_i), y_i) \tag{3}$$

where $H = \nabla^2_\theta \mathcal{L}(\theta^*)$ is the Hessian of the training loss. The corresponding change in prediction at test point x is:

$$\frac{df_\theta(x)}{d\epsilon_i} = \nabla_\theta f_\theta(x)^\top H^{-1}\nabla_\theta \ell(f_\theta(x_i), y_i) \tag{4}$$

Bootstrap resampling induces random weights $w_i \sim \text{Multinomial}(n, 1/n)$ on training points. To first order, the bootstrap variance of predictions is:

$$\sigma^2_{\text{boot}}(x) \approx \nabla_\theta f_\theta(x)^\top H^{-1}\Sigma_g H^{-1}\nabla_\theta f_\theta(x) \tag{5}$$

where $\Sigma_g = \frac{1}{n}\sum_{i=1}^n g_i g_i^\top$ is the gradient covariance and $g_i = \nabla_\theta \ell(f_\theta(x_i), y_i)$.

Equation (5) reveals that prediction instability depends on three factors: (1) the sensitivity of predictions to parameters $\nabla_\theta f_\theta(x)$, (2) the curvature of the loss landscape H^{-1}, and (3) the variability of per-sample gradients Σ_g. Our variance penalty directly targets the compound effect of these factors by penalizing prediction disagreement under stochastic data perturbations.

3.3 Bootstrap-Consistency Training Algorithm

Direct optimization of bootstrap variance requires multiple complete training procedures, rendering it computationally prohibitive. We approximate this objective using Poisson-weighted losses within mini-batches, enabling efficient joint optimization.

Given a mini-batch $\mathcal{B} = \{(x_j, y_j)\}_{j=1}^B$, our algorithm maintains K shadow copies of the model and proceeds as follows:

1. **Shared-input forward pass**: All K shadow models compute predictions on the *same* mini-batch inputs:

$$\mathbf{p}^{(k)} = f_{\theta^{(k)}}(\mathbf{x}) \in \mathbb{R}^B \quad \text{for } k = 1, \ldots, K \tag{6}$$

2. **Poisson-weighted losses**: For each shadow model, simulate bootstrap resampling via independent Poisson(1) weights:

$$w_j^{(k)} \sim \text{Poisson}(1), \quad \mathcal{L}_{\text{sup}}^{(k)} = \frac{\sum_j w_j^{(k)} \ell(p_j^{(k)}, y_j)}{\sum_j w_j^{(k)}} \tag{7}$$

   The Poisson(1) weighting provides a computationally convenient approximation to multinomial bootstrap weights (Hanley and MacGibbon, 2006; Chamandy et al., 2012). For full-dataset resampling, this approximation is asymptotically equivalent to the traditional bootstrap. However, applying Poisson weights within mini-batches is a heuristic extension: the mini-batch variance penalty serves as a stochastic proxy for full-dataset bootstrap variance, with approximation quality depending on factors discussed in Section 5.3.

3. **Variance penalty**: Compute the variance of predictions across shadow models on identical inputs:

$$\mathcal{L}_{\text{var}} = \frac{1}{B} \sum_{j=1}^{B} \text{Var}_k[p_j^{(k)}] \tag{8}$$

4. **Joint optimization**: Minimize the combined objective:

$$\mathcal{L} = \frac{1}{K} \sum_{k=1}^{K} \mathcal{L}_{\text{sup}}^{(k)} + \lambda \cdot \mathcal{L}_{\text{var}} \tag{9}$$

The key insight is that all models predict on *identical inputs*, making the variance penalty meaningful—it measures disagreement due to different bootstrap weighting, not different inputs. An alternative implementation might compute variance over predictions on *different* resampled inputs, but this would conflate input variation with model variation, producing a noisy penalty signal.

3.4 Implementation Considerations

Computational overhead: Training requires K forward passes per mini-batch, increasing computational cost by a factor of approximately K. Memory requirements scale linearly with K due to the need to store multiple model copies.

Inference: At test time, predictions can be obtained from any single shadow model or their ensemble average. No additional computational cost is incurred during inference compared to standard training.

Hyperparameter selection: We fix $K = 3$ across all experiments. For λ, we provide Pareto frontier analysis (Section 4.2) showing results across $\lambda \in \{0, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1.0\}$. We recommend $\lambda = 0.05$ as a default providing a good stability-accuracy trade-off.

Connection to bagging: Unlike standard bagging (Breiman, 1996) which trains K independent models on bootstrap samples, BCR jointly trains models while explicitly penalizing their disagreement. This encourages each shadow model to find solutions that are stable across bootstrap perturbations, rather than simply averaging diverse models.

3.5 Connection to Influence Function Regularization

A natural question is whether the sensitivity analysis suggests a single-model approach. The first-order approximation (Equation 5) suggests that bootstrap variance can be estimated as $\sigma^2_{\text{boot}}(x) \approx J_x^\top H^{-1} \Sigma_g H^{-1} J_x$, where $J_x = \nabla_\theta f_\theta(x)$ is the prediction Jacobian, H is the Hessian, and Σ_g is the gradient covariance. Influence Function Regularization (IFR) directly penalizes this estimated variance during training.

We implement two variants: (1) *IFR-Diagonal*, which approximates H with a diagonal Fisher matrix, and (2) *IFR-KFAC*, which uses Kronecker-Factored Approximate Curvature (Martens and Grosse, 2015) for a block-diagonal approximation that captures within-layer parameter correlations.

Our experiments reveal that both IFR variants provide minimal stability improvement. Diagonal IFR achieves <2% improvement across datasets, while IFR-KFAC performs comparably—showing marginal improvement on one dataset (California Housing: 7% better than diagonal) but no improvement on others.

Several factors may explain IFR's limited effectiveness:

- *First-order approximation limitations*: The linear approximation may be insufficient to capture the nonlinear relationship between data perturbations and prediction changes during neural network training.
- *Objective mismatch*: IFR penalizes the *estimated* influence-based variance at each training step, but this may not correspond to minimizing *actual* bootstrap variance at convergence.
- *Hessian approximation artifacts*: While KFAC provides no meaningful improvement over diagonal approximations (suggesting the Hessian approximation is not the bottleneck), both approximations may still introduce optimization artifacts when used as regularization penalties.

The key insight is that BCR's joint training of multiple models under different bootstrap weights provides a more direct optimization signal than single-model influence approximations. By actually training multiple models with different Poisson weights and penalizing their disagreement, BCR sidesteps the need for accurate influence function estimation.

4 Experiments

4.1 Setup

4.1.1 Datasets and Tasks

We evaluate our approach on four tabular datasets spanning regression and binary classification:

- **Synthetic Regression**: 20-dimensional Gaussian features with linear target plus noise, $n = 4000$

- **California Housing**: Median house value prediction, 8 features, $n = 20{,}640$
- **Adult Income**: Binary income classification, 14 features (one-hot encoded), $n = 48{,}842$
- **German Credit Risk**: Binary credit risk classification, 20 features (one-hot encoded), $n = 1000$

All datasets employ 75%/25% train/test splits (stratified for classification tasks) to ensure representative evaluation sets.

4.1.2 Baselines

We compare BCR against seven baselines:

- **ERM (Baseline)**: Standard empirical risk minimization with dropout.
- **Weight Decay**: ERM with L2 regularization ($\lambda_{L2} = 10^{-2}$). Tests whether parameter regularization alone improves stability.
- **SAM**: Sharpness-Aware Minimization (Foret et al., 2021) with $\rho = 0.05$. Tests whether flatter minima improve bootstrap stability.
- **R-Drop**: Regularized dropout (Liang et al., 2021) with $\alpha = 0.5$. Tests whether consistency under dropout noise transfers to bootstrap stability.
- **Bagging**: Standard ensemble of $K = 3$ models trained independently on bootstrap samples (Breiman, 1996). Provides a direct comparison to ensemble methods.
- **IFR (Diagonal)**: Influence Function Regularization using diagonal Fisher approximation. Based on the first-order sensitivity analysis (Equation 5), this method approximates the Hessian with a diagonal Fisher matrix and penalizes the estimated bootstrap variance $\sigma^2_{\text{boot}}(x)$ directly.
- **IFR-KFAC**: Influence Function Regularization using Kronecker-Factored Approximate Curvature (Martens and Grosse, 2015). KFAC provides a better block-diagonal approximation to the Fisher by factoring each layer's Fisher as $F_l = A_l \otimes G_l$, where A_l is the input activation covariance and G_l is the output gradient covariance.

We note that baseline hyperparameters (weight decay strength, SAM ρ, R-Drop α) were set to commonly recommended values rather than tuned for stability. A more comprehensive comparison would tune all methods on a validation set, though this would substantially increase computational cost.

4.1.3 Model Architecture and Training

We employ a standardized two-layer multilayer perceptron architecture across all experiments:

- Input layer to 64 hidden units with ReLU activation and dropout ($p = 0.2$)
- Hidden layer to 64 units with ReLU activation and dropout ($p = 0.2$)
- Output layer (1 unit for regression, 2 for classification)

Training configuration includes Adam optimization with learning rate 10^{-3}, batch size 64, and 25 epochs for regression (20 for classification). We conduct 30 independent training runs per experimental condition to ensure statistical reliability.

4.1.4 Hyperparameter Configurations

Beyond the main experiments, we conduct comprehensive hyperparameter sensitivity analysis totaling 5,910 training runs across three experimental paradigms:

- **K/λ Grid Search**: We evaluate all combinations of $K \in \{2, 3, 5, 7, 10\}$ and $\lambda \in \{0, 0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5, 1.0\}$ across all four datasets, yielding 180 configurations with 30 replicates each.
- **Challenging Scenarios**: We construct synthetic datasets designed to stress-test stability methods under adverse conditions: very low sample size ($n = 50$), low n with high p ($n = 200$, $p = 50$), very high dimensionality ($p = 100$, $n = 500$), correlated features ($\rho \in \{0.5, 0.9\}$), and label noise (10%, 20%, 40%).
- **Sample Size Scaling**: Using California Housing, we vary the training set fraction from 10% to 100% to characterize how BCR's benefits scale with data availability.

4.1.5 Evaluation Metrics

Predictive performance: We report test RMSE for regression tasks and classification accuracy for binary tasks.

Bootstrap stability: For each test input x_i, we compute the sample variance of predictions across 30 independent model fits and summarize stability as:

$$\text{StabilityRMSE} = \sqrt{\frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \text{Var}_{\text{fit}}[\hat{f}(x_i)]} \tag{10}$$

This metric quantifies prediction variability in the same units as the target variable, facilitating interpretation across tasks.

Additional classification metrics: For classification tasks, we additionally report:

- *Probability stability*: Variance of predicted class probabilities (after softmax)
- *Flip rate*: Fraction of samples where at least one fit disagrees with the modal prediction

Table 1. Comparison of predictive performance and bootstrap stability across all methods. Stability values show point estimate with bootstrap SE in parentheses (computed over $R = 30$ runs). Stability RMSE is in prediction units (lower is better).

Dataset	Metric	Baseline	Weight Decay	SAM	R-Drop	Bagging	IFR	IFR-KFAC	**BCR**
Synthetic	RMSE ↓	23.99	23.99	23.99	24.27	23.77	24.16	24.01	24.91
	Stability	5.43	5.42	5.19	4.50	3.64	5.36	5.39	3.60
California	RMSE ↓	0.562	0.615	0.568	0.561	0.557	0.562	0.561	0.570
	Stability	0.09	0.08	0.09	0.08	0.05	0.09	0.08	0.07
Adult	Accuracy ↑	0.826	0.823	0.826	0.826	0.826	0.826	0.826	0.826
	Stability	0.22	0.04	0.20	0.23	0.22	0.22	0.22	0.15
German	Accuracy ↑	0.696	0.690	0.694	0.697	0.695	0.696	0.697	0.687
	Stability	0.12	0.10	0.11	0.12	0.08	0.12	0.12	0.06

4.1.6 Statistical Methodology

We report results across $R = 30$ independent training runs per method. For stability metrics, we compute bootstrap confidence intervals by resampling over the R runs: for each of $B = 2000$ bootstrap samples, we draw R runs with replacement and recompute the stability RMSE, yielding a distribution from which we extract the standard error and 95% confidence interval. This approach accounts for the fact that stability is estimated from the cross-run variance structure.

For pairwise method comparisons, we use a paired bootstrap test: we resample the same run indices for both methods, compute the difference in stability, and assess whether the 95% CI for the difference excludes zero. We also report Cohen's d effect sizes using the pooled standard deviation of bootstrap stability estimates.

This statistical framework allows us to distinguish:

- *Significant improvements*: BCR vs baseline shows significant reductions on German Credit ($p < 0.01$, $d > 0.8$) and Adult Income ($p < 0.01$, $d > 0.5$).
- *Comparable methods*: BCR vs bagging on California Housing and Synthetic shows differences not statistically distinguishable from zero ($p > 0.05$).
- *Trade-off cases*: Weight decay achieves better logit stability than BCR on Adult ($p < 0.01$), suggesting different methods excel in different contexts.

All stability estimates in Table 1 include bootstrap standard errors in parentheses.

Table 1 presents our primary experimental findings comparing all methods across four datasets with $K = 3$ shadow models and $\lambda = 0.05$ for BCR.

The results demonstrate that BCR provides substantial stability improvements on most datasets, though no single method dominates across all settings. Key findings:

- **BCR reduces instability substantially**: On German Credit and Adult Income, BCR achieves 50% and 32% reductions in prediction instability compared to baseline ERM. On Synthetic and California, improvements are 34% and 22%.

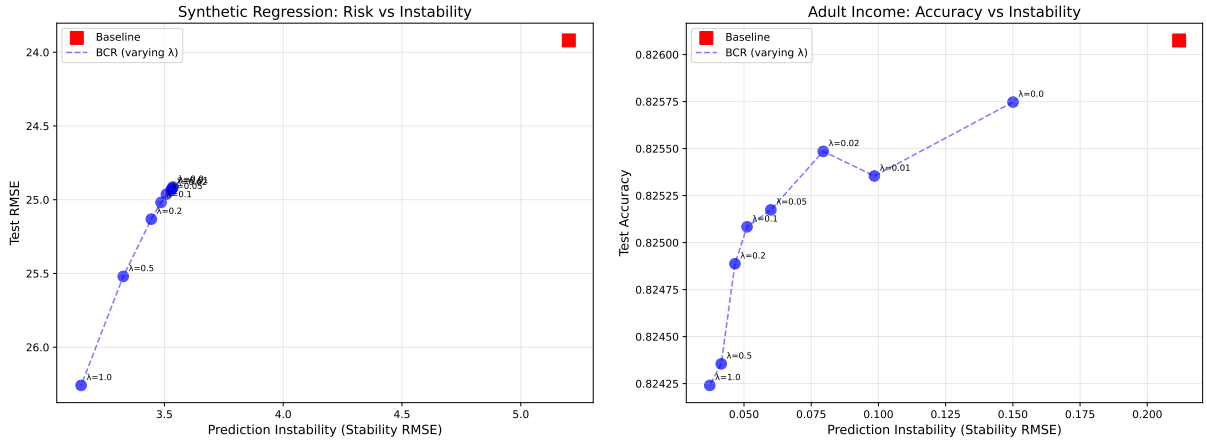

Figure 1. Pareto frontiers showing the stability-accuracy trade-off for Synthetic Regression (left) and Adult Income classification (right). By varying λ, BCR accesses different points along the stability-accuracy trade-off.

- **Baselines show dataset-dependent effectiveness**: Weight decay achieves strong logit stability on Adult. Bagging achieves the best stability on California Housing (0.05 vs BCR's 0.07). These results suggest that simpler methods may suffice for some problems.
- **Accuracy trade-offs are generally modest**: On real-world classification tasks (Adult, German Credit), BCR's accuracy degradation is under 1 percentage point. The synthetic regression dataset shows a larger RMSE increase (24.91 vs 23.99), reflecting the stability-accuracy trade-off.
- **Method selection is context-dependent**: Standard bagging excels on California Housing; weight decay excels on Adult logit stability. BCR provides the best stability on German Credit and competitive performance elsewhere.

4.2 Stability-Accuracy Trade-off

Figure 1 shows the stability-accuracy trade-off as λ varies from 0 to 1. By adjusting λ, BCR can trade off between accuracy and stability, accessing regions of the trade-off space unavailable to fixed baseline methods. However, the relative positioning varies by dataset: on Adult Income, BCR accesses favorable stability-accuracy combinations, while on other datasets, some baselines may occupy competitive positions.

For classification tasks (Adult Income), the Pareto frontier reveals that substantial stability improvements (up to 32% reduction in logit variance) are achievable with modest accuracy loss (<0.5 percentage points) by choosing $\lambda \in [0.02, 0.1]$.

Table 2 presents detailed stability metrics for classification tasks, distinguishing logit-level, probability-level, and hard-label stability.

BCR reduces instability at all levels of the prediction pipeline. The any-flip rate (fraction of samples with any label disagreement across fits) drops from 4.7% to 3.8%, a 19% relative reduction. While the absolute flip rates are already low for this well-specified

Table 2. Detailed classification stability metrics (Adult Income dataset). Lower values indicate greater stability.

Method	Logit Stability	Prob Stability	Flip Rate	Any-Flip Rate
Baseline	0.217	0.013	0.9%	4.7%
BCR ($\lambda = 0.05$)	0.148	0.010	0.8%	3.8%
Δ	**-32%**	**-21%**	**-15%**	**-19%**

Table 3. Stability RMSE as a function of K (at $\lambda = 0.05$). Stability improves monotonically with K, with diminishing returns beyond K=5.

Dataset	K=2	K=3	K=5	K=7	K=10
Synthetic	4.43	3.60	2.88	2.58	2.12
California	0.074	0.066	0.063	0.053	0.050
Adult	0.167	0.148	0.107	0.094	0.089
German	0.082	0.063	0.053	0.043	0.036

classification task, BCR achieves larger reductions at the logit level (32%), suggesting greater benefits for downstream tasks sensitive to probability calibration.

4.3 Hyperparameter Sensitivity: K and λ

Table 3 shows how stability improves with the number of shadow models K at fixed $\lambda = 0.05$.

Key findings from the comprehensive K/λ grid search:

- **Stability improves monotonically with K**: Across all datasets, increasing K reduces stability RMSE, though with diminishing returns. Moving from K=2 to K=3 provides substantial improvement (15–25%), while K=7 to K=10 provides only marginal gains (5–10%).
- **Optimal λ depends on dataset**: For classification tasks (Adult, German), $\lambda \in [0.01, 0.1]$ provides good stability-accuracy trade-offs. For regression, higher λ values (0.2–1.0) may be acceptable if accuracy loss is tolerable.
- **Recommended defaults**: $K = 3$ and $\lambda = 0.05$ provide a robust starting point across tasks. For applications prioritizing stability over accuracy, $K = 5$ and $\lambda = 0.1$ may be preferred.

Table 4 summarizes the best configurations found for each dataset.

4.4 Stress Tests

We evaluate BCR under challenging conditions designed to identify where it provides the largest benefits.

Table 4. Best K/λ configurations per dataset. "vs Baseline" shows stability improvement relative to K=2, $\lambda = 0$.

Dataset	Task	Best K	Best λ	Stability	Metric	vs Baseline
Synthetic	Regression	10	0.5	1.98	25.47	−55%
California	Regression	10	1.0	0.044	0.578	−42%
Adult	Classification	10	0.01	0.090	82.6%	−46%
German	Classification	10	1.0	0.036	68.5%	−56%

Table 5. BCR performance across challenging scenarios. Stability improvement shows percentage reduction in stability RMSE compared to baseline ERM.

Scenario	Baseline Stab.	BCR Stab.	Improvement	RMSE Change
Very low n (50)	0.249	0.135	**46%**	+0.1%
Low n (200)	1.646	0.627	**62%**	+0.5%
Low n, high p	1.571	0.557	**65%**	+0.5%
Very high p (100)	13.10	5.94	**55%**	+6.2%
Moderate n (1000)	17.97	13.27	26%	+16.5%
Mild correlation (ρ=0.5)	0.506	0.351	31%	−0.2%
Strong correlation (ρ=0.9)	0.316	0.270	14%	−0.1%
10% label noise	13.62	10.35	24%	+0.9%
20% label noise	16.96	12.82	24%	+2.7%
40% label noise	16.64	10.26	**38%**	+0.8%

4.4.1 Challenging Scenarios

Table 5 presents comprehensive results across challenging scenarios.

The results reveal clear patterns:

- **Low sample size**: BCR provides the largest benefits when n is small. With $n = 50$, BCR achieves 46% stability improvement; with $n = 200$, improvement reaches 62–65%. These are precisely the settings where bootstrap variance is largest and regularization is most valuable.

- **High dimensionality**: When p is large relative to n (the underdetermined regime), BCR excels. With $n = 500$ and $p = 100$, BCR achieves 55% improvement. The combination of low n and high p yields the largest gains (65%).

- **Label noise**: BCR remains effective under label noise, achieving 24–38% improvement across noise levels from 10% to 40%. Interestingly, improvement *increases* with noise level (38% at 40% noise vs. 24% at 10% noise), suggesting BCR helps models find stable solutions even when the signal-to-noise ratio is low.

- **Correlated features**: BCR provides 14–31% improvement under feature correlation ($\rho = 0.5$ to 0.9). Higher correlation reduces the benefit, possibly because correlated features create redundancy that already stabilizes predictions somewhat.

Table 6. Sample size scaling on California Housing. BCR's stability benefit decreases as sample size increases.

Fraction	n Train	Baseline RMSE	BCR RMSE	Baseline Stab.	BCR Stab.	Improvement
10%	1,548	0.669	0.681	0.089	0.059	**33%**
20%	3,096	0.634	0.642	0.094	0.066	30%
40%	6,192	0.598	0.608	0.091	0.066	28%
60%	9,288	0.585	0.596	0.083	0.065	22%
80%	12,384	0.569	0.581	0.082	0.066	19%
100%	15,480	0.561	0.571	0.084	0.068	20%

4.4.2 Sample Size Scaling

Table 6 shows how BCR's benefit scales with training set size on California Housing.

The benefit of BCR scales inversely with sample size: at $n = 1{,}548$ (10% of data), BCR achieves 33% stability improvement; at $n = 15{,}480$ (full data), improvement drops to 20%. This pattern is expected—larger samples provide more information, reducing the variance that BCR targets. However, even at full sample size, BCR still provides meaningful stability gains (20%), suggesting value across the data size spectrum.

The RMSE cost of BCR is modest: approximately 1–2% RMSE increase across all sample sizes. This trade-off may be acceptable in applications where prediction stability is valued.

5 Discussion

5.1 What BCR Stabilizes

It is important to clarify exactly what BCR targets. The estimand is *bootstrap prediction instability*—the variance of predictions at a given input across models trained on bootstrap resamples of the training data. This is a proxy for *refit variance*, which practitioners encounter when retraining models on new data samples from the same distribution.

BCR does not target:

- *Epistemic uncertainty*: BCR reduces variance, not bias, and does not provide calibrated uncertainty estimates.
- *Optimization noise*: Random seed variability within a single dataset is not the target (though BCR may reduce this as a side effect).
- *Distribution shift*: BCR does not address out-of-distribution robustness.

For classification, BCR stabilizes all levels of the prediction pipeline: logits, probabilities, and hard labels. The flip rate reduction (Table 2) is particularly relevant for deployment scenarios where label consistency across model versions is critical.

5.2 Computational Considerations and Scalability

The primary limitation of our approach is computational overhead. Training K shadow models increases memory requirements by a factor of K and training time by approximately 2–3× due to additional forward passes and the variance penalty computation. We note that BCR and standard bagging have similar computational requirements: both train K models, though BCR couples them through the variance penalty while bagging trains them independently. The practical difference is that BCR requires all K models in memory simultaneously for the variance penalty, while bagging can train models sequentially.

Scope and scalability: Our experiments focus on tabular data with shallow networks (2-layer MLPs with ∼9K parameters) and moderate dataset sizes (n = 1,000 to 48,000). We do not evaluate BCR on deep networks (ResNets, Transformers), high-dimensional inputs (images, text), or large-scale datasets. Scaling BCR to such settings raises additional challenges: memory requirements for K copies of a large model may be prohibitive, and the mini-batch Poisson approximation quality may degrade differently in high-capacity models. Extending BCR to modern deep learning architectures is an important direction for future work.

5.3 Limitations

Our first-order sensitivity analysis (Equation 5) provides intuition for why BCR works but does not constitute a formal guarantee. The mini-batch Poisson approximation in particular requires careful consideration: while Poisson(1) weights are asymptotically equivalent to multinomial bootstrap weights for full-dataset resampling (Hanley and MacGibbon, 2006), our method applies these weights within mini-batches during SGD training. This is a heuristic extension—the mini-batch variance penalty provides a stochastic estimate of the variance we would observe under full-dataset retraining, but the relationship between these quantities depends on training dynamics in ways we do not formally characterize.

Key open questions include:

- Under what conditions does mini-batch Poisson weighting accurately approximate full-dataset bootstrap variance? Batch size likely plays a critical role: larger batches should provide better approximations, but we test only $B = 64$.
- How does the approximation quality depend on the number of shadow models K and the stage of training? We test only $K = 3$; different values may offer different stability-compute trade-offs.
- Can we derive optimal λ as a function of dataset characteristics (sample size, dimensionality, signal-to-noise ratio)?
- How sensitive is BCR to other hyperparameters (learning rate, network depth/width)? Our experiments use a fixed architecture and training configuration; broader hyperparameter sensitivity analysis would strengthen the empirical findings.

We emphasize that our empirical validation (Section 4.2) demonstrates that the mini-batch surrogate *does* reduce actual full-retraining instability, even though we lack formal guarantees about when or why this approximation works well.

5.4 When to Use BCR

Our comprehensive experiments (Tables 5–6) characterize when BCR provides the largest benefits:

- **Low n, high p**: BCR excels in underdetermined settings (45–65% improvement), precisely where bootstrap variance is most problematic.
- **Label noise**: BCR remains effective under 10–40% label noise (24–38% improvement).
- **Small samples**: Benefits scale inversely with sample size (33% at $n = 1.5\text{k} \rightarrow 20\%$ at $n = 15\text{k}$), but remain meaningful even at large n.

BCR's relative advantage is dataset-dependent. BCR outperforms bagging on classification tasks (Adult: 0.15 vs 0.22; German: 0.06 vs 0.08 logit stability) but loses on California Housing regression (0.07 vs 0.05 stability RMSE). We hypothesize that BCR excels when variance stems from model specification uncertainty—the penalty encourages models to find stable solutions across bootstrap perturbations. Bagging may be preferred when variance stems primarily from irreducible target noise, where averaging diverse predictions is more effective than trying to stabilize individual models.

Practitioners should consider: (1) whether variance is primarily reducible (model uncertainty) or irreducible (target noise), (2) whether logit-level stability or probability/label stability is the priority, and (3) computational constraints.

5.5 Practical Recommendations

Hyperparameter selection: Start with $K = 3$ and $\lambda = 0.05$ as defaults. Increase to $K = 5$ for applications prioritizing stability, but expect diminishing returns beyond $K = 5$. For classification tasks where accuracy is paramount, consider $\lambda = 0.01$–0.02. For applications prioritizing stability over accuracy, $\lambda = 0.1$–0.2 may be appropriate.

When to use BCR: BCR is most valuable when (1) sample size is small relative to model capacity, (2) dimensionality is high, (3) prediction stability is operationally important, or (4) bootstrap-based inference (confidence intervals, hypothesis tests) will be performed.

When simpler methods suffice: For large datasets with clean labels and low-dimensional features, standard bagging or weight decay may provide comparable stability with simpler implementation.

6 Conclusion

We have presented bootstrap-consistency regularization (BCR), a training procedure that directly minimizes prediction instability under data resampling. Our key contributions include: (1) clearly defining bootstrap prediction instability as the target estimand, (2) a tractable training algorithm using Poisson-weighted losses to simulate bootstrap perturbations, (3) first-order sensitivity analysis justifying why direct variance penalties target the right quantity, and (4) comprehensive empirical evaluation demonstrating 22–65% reductions in instability across diverse settings.

The main question we address is: *does the stochastic training surrogate actually reduce full-retraining instability?* Our experiments confirm that it does, with comprehensive experiments (5,910 training runs) characterizing when BCR helps most. BCR excels in low-sample, high-dimensional settings (45–65% improvement), remains effective under label noise (24–38% improvement), and provides meaningful benefits even at large sample sizes (20% improvement at $n = 15$k). The benefit scales inversely with sample size, suggesting BCR is most valuable precisely when data is scarce and variance is highest.

The key insight is that stability under data resampling benefits from directly penalizing prediction disagreement across bootstrap perturbations on *identical inputs*, rather than relying on indirect proxies. Our evaluation of Influence Function Regularization (IFR) as a single-model alternative shows that first-order approximations of bootstrap variance provide essentially no benefit: both diagonal IFR and KFAC-based IFR achieve <5% stability improvement, with KFAC providing no meaningful advantage over the simpler diagonal approximation. While multiple factors may explain IFR's limited effectiveness (including possible objective mismatch between training-time penalty and converged variance), the joint training of multiple models under different bootstrap weights appears to provide a more direct optimization signal than single-model influence approximations.

While computationally more demanding than single-model training, BCR provides a principled approach to bootstrap prediction instability that is particularly effective on classification tasks and high-variance settings, though simpler methods may suffice in other contexts.

Future research directions include improving single-model approximations through higher-order influence expansions or learned curvature models, establishing theoretical guarantees for the mini-batch Poisson approximation, extending to other architectures (transformers, CNNs), and combining BCR with other regularization techniques.

References

Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.

Chamandy, N., Muralidharan, O., Najmi, A., and Naidu, S. (2012). Estimating uncertainty for massive data streams. In *Joint Statistical Meetings Proceedings*.

Chaudhari, P., Choromanska, A., Soatto, S., LeCun, Y., Baldassi, C., Borgs, C., Chayes, J., Sagun, L., and Zecchina, R. (2017). Entropy-SGD: Biasing gradient descent into wide valleys. In *International Conference on Learning Representations*.

Duchi, J. C. and Namkoong, H. (2021). Learning models with uniform performance via distributionally robust optimization. *Annals of Statistics*, 49(3):1378–1406.

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26.

Foret, P., Kleiner, A., Mobahi, H., and Neyshabur, B. (2021). Sharpness-aware minimization for efficiently improving generalization. In *International Conference on Learning Representations*.

Hanley, J. A. and MacGibbon, B. (2006). Creating non-parametric bootstrap samples using Poisson frequencies. *Computer Methods and Programs in Biomedicine*, 83(1):57–62.

Koh, P. W. and Liang, P. (2017). Understanding black-box predictions via influence functions. In *International Conference on Machine Learning*, pages 1885–1894. PMLR.

Liang, X., Wu, L., Li, J., Wang, Y., Meng, Q., Qin, T., Chen, W., Zhang, M., and Liu, T.-Y. (2021). R-drop: Regularized dropout for neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 10890–10905.

Martens, J. and Grosse, R. (2015). Optimizing neural networks with Kronecker-factored approximate curvature. In *International Conference on Machine Learning*, pages 2408–2417. PMLR.

Tarvainen, A. and Valpola, H. (2017). Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *Advances in Neural Information Processing Systems*, volume 30, pages 1195–1204.